ECON 5015 Midterm Cram Guide

DataFrames first, then everything else

0. What the exam looks like (from both practice midterms)

Roughly 50 points, about 8 questions. The same question types repeat:

Question type	What to have ready
Short loop fragment, no built-ins (5 pts)	Accumulator template (§4): factorial, product, sum, min/max.
Small function with dict/list (5–6 pts)	`//` and `%`; return a dict; build a new list with `append`; don't modify the input.
DataFrame manipulation (6 pts)	Long→wide reshape without `pivot` (§1.9).
Implement an algorithm from pseudocode (8 pts)	Translate line by line (§9); insertion sort; grid search; O and Ω.
Scope/mutability trace (8 pts ×2)	The 5-step trace method (§2). Watch for dead code after `return`.
Big-O of given code (4 pts)	Loops multiply; halving gives $\log n$; early `return` gives $\Omega(1)$ (§8).

"No built-in functions" means you can use only `len`, `range`, operators, and list methods (`append`, `copy`, …). No `sum`, `max`, `min`, `sorted`, `abs`.

1. DataFrames (pandas) — your weak spot, so the most detail

1.1 Mental model

- A **Series** is a 1-D array *plus an index* (row labels). The default index is `0,1,2,...`.
- A **DataFrame** is a set of Series (the columns) that all share *one* index. Each column has a single type, and different columns can have different types.
- A DataFrame breaks into three pieces: `df.values` (a 2-D NumPy array), `df.columns` (the column names), and `df.index` (the row labels).
- DataFrames are **mutable**: `df2 = df` is the *same* object. Use `df.copy()` for an independent copy.

1.2 Creating and importing

```
import numpy as np
import pandas as pd
df = pd.DataFrame({"numbers": [1, 2, 3], "letters": ["a", "b", "c"]})  # dict: column name -> list/array
df = pd.read_csv("aapl.csv")          # filename only, file sits next to the script
s  = pd.Series([1, 3, 5])             # default index 0,1,2
s2 = pd.Series(["a", "b"], ["A", "B"])  # data, then index
```

1.3 Getting columns

```
df["letters"]          # one column -> Series   (always works)
df.letters             # same thing             (only if the name has no spaces and isn't a method name)
df[["numbers", "letters"]]   # list of names -> DataFrame
df.columns             # Index of names; df.columns[1] -> "letters"
```

1.4 Getting rows and cells: `iloc` vs `loc` (this is the core idea)

iloc = position, loc = label

`iloc` counts positions `0..n-1`, and slices *exclude* the end (like lists).
`loc` uses index *labels*, and slices *include* the end.
With a fresh default index, labels equal positions, so both look the same. They differ **after you filter, drop, or sort rows**.

```
df.iloc[0]            # first row (as a Series)
df.iloc[1:4]          # rows at positions 1,2,3 (DataFrame)
df.iloc[2:3]          # one row, kept as a DataFrame
df.iloc[2, 1]         # row position 2, column position 1
df.loc[3, "letters"]  # row LABEL 3, column "letters"
df.loc[1:3]           # labels 1,2,3  (end INCLUDED)
df.letters.iloc[0]    # first value of a column   <- the safest way to get one cell
df.values[2, 1]       # via the NumPy array: always positional
df.values[:, 1]       # whole column 1 as an array
```

Trap: `df.col[i]` uses the LABEL, not the position

```
b = df[df.week > 2]   # keeps labels 2,3  (NOT 0,1)
b.x[2]                # works: label 2
b.x[0]                # KeyError: no label 0
b.x.iloc[0]           # works: first row by position
```

If you loop with `for i in range(len(df))`, use `.iloc[i]` or `.values[i]`. Then the loop works even when the index isn't `0..n-1`.

1.5 Filtering rows (boolean masks)

```
df[df.numbers > 1]                          # rows where condition is True
df[(df.week >= 80) & (df.week <= 85)]       # AND: use &, and PARENTHESES around each condition
df[(df.x < 0) | (df.y > 5)]                 # OR: |      NOT: ~(df.x > 3)
df[df.borough.isin([1, 2, 3, 6, 14])]       # value is in a list
df[df.week < 53].reset_index(drop=True)     # filter, then renumber rows 0..n-1
```

Trap: use `&`, `|`, `~` on Series, never `and`/`or`/`not`

`(df.x > 1) and (df.x < 5)` raises "truth value of a Series is ambiguous". You also need the parentheses, because `&` binds tighter than `>`.

1.6 Adding, changing, and removing

```
df["neg"]  = np.arange(-1, -4, -1)          # new column from an array (length must match)
df["mult"] = df.numbers * df.neg            # vectorized column math, row by row
df["crimerate"] = df["crime"] / df["population"]
df["lcrime"] = np.log(df["crimerate"])      # numpy functions work element-wise on columns
df["treat"] = (df.week >= 80).astype(int)   # bool -> 0/1 dummy
df = df.drop("mult", axis="columns")        # drop a column  (returns NEW df -> reassign!)
df = df.drop(3)                             # drop the row with LABEL 3
df = df.reset_index(drop=True)              # relabel rows 0..n-1 (drop=True: don't keep old index as a column)
new = pd.DataFrame({"numbers": [27], "letters": ["aa"], "neg": [-27]})
df = pd.concat([df, new], ignore_index=True)   # add rows (concat makes a new df)
df = df.sort_values(["borough", "week"])    # sort by borough, then by week
```

Trap: `drop`, `concat`, `sort_values`, `reset_index` return a NEW DataFrame

Calling `df.drop(3)` on its own line changes nothing. Write `df = df.drop(3)`.

1.7 Summary statistics (HW3 style: "use numpy functions")

```
high = np.max(df.High)        # highest High
low  = np.min(df.Low)
avg  = np.mean(df.Close)
sd   = np.std(df.Close)       # POPULATION sd (ddof=0).  df.Close.std() is SAMPLE sd (ddof=1)
date = df.Date.iloc[np.argmax(df.Volume)]   # date on the highest-volume day (argmax = POSITION)
np.sum(df.x > 3)              # how many rows satisfy the condition (True counts as 1)
len(df)                       # number of rows;   df.shape -> (rows, cols)
```

Trap: footnote rows in CSVs

`yhoo.csv` has a footnote on the last row. The class fix was `yhoo = yhoo.drop(66)` (the label of that row). Check `df.tail()` before computing anything.

1.8 Index alignment: the trap from HW4 (differencing)

Arithmetic between two Series lines up rows *by index label*, not by position.

```
df1 = df[df.week < 53]          # labels 0..k
df2 = df[df.week > 52]          # labels k+1..  -> no labels overlap with df1!
df2.x - df1.x                   # ALL NaN  (no matching labels)
df1 = df[df.week < 53].reset_index(drop=True)
df2 = df[df.week > 52].reset_index(drop=True)
df2.x - df1.x                   # correct: row i minus row i
df2.x.values - df1.x.values     # also correct: plain arrays ignore the index
```

HW4 recipe: sort by `[borough, week]`, split into the two years, `reset_index(drop=True)` on both, then subtract column by column into a new DataFrame. This works because both halves are in the same borough/week order.

1.9 Reshaping long → wide WITHOUT `pivot` (practice midterm Q4)

Setup: `long` has 4000 rows (columns `id, year, outcome`); each firm has 4 consecutive rows, years 1 to 4. You need `wide` with columns `id, year1, ..., year4`, one row per firm.

Idea: firm k's rows are at positions $4k, 4k+1, 4k+2, 4k+3$. Walk through in steps of 4 and collect the values into lists.

```
ids = []; y1 = []; y2 = []; y3 = []; y4 = []
i = 0
while i < len(long):                      # or: for i in range(0, len(long), 4):
    ids.append(long.id.iloc[i])
    y1.append(long.outcome.iloc[i])
    y2.append(long.outcome.iloc[i + 1])
    y3.append(long.outcome.iloc[i + 2])
    y4.append(long.outcome.iloc[i + 3])
    i = i + 4
wide = pd.DataFrame({"id": ids, "year1": y1, "year2": y2, "year3": y3, "year4": y4})
```

Faster version with slicing (step-4 slices pick every 4th row; no loop needed):

```
v = long.outcome.values
wide = pd.DataFrame({"id": long.id.values[::4], "year1": v[0::4], "year2": v[1::4],
                     "year3": v[2::4], "year4": v[3::4]})
```

If the rows were NOT in order, map each id to a list of 4 slots with a dict:

```
d = {}
for k in range(len(long)):
    fid = long.id.iloc[k]
    if fid not in d:
        d[fid] = [0, 0, 0, 0]
    d[fid][long.year.iloc[k] - 1] = long.outcome.iloc[k]   # year 1 -> slot 0
ids = list(d.keys())
wide = pd.DataFrame({"id": ids, "year1": [d[f][0] for f in ids], "year2": [d[f][1] for f in ids],
                     "year3": [d[f][2] for f in ids], "year4": [d[f][3] for f in ids]})
```

Reverse (wide → long): a nested loop over firms and years.

```
ids = []; yrs = []; outs = []
for k in range(len(wide)):
    for t in range(1, 5):
        ids.append(wide.id.iloc[k]); yrs.append(t); outs.append(wide["year" + str(t)].iloc[k])
long = pd.DataFrame({"id": ids, "year": yrs, "outcome": outs})
```

Every snippet above was run and checked against the practice example (firms 100 and 104).

1.10 Other pandas you might see

```
df.head(); df.tail(); df.shape; df.dtypes
df.groupby("id").outcome.mean()           # mean outcome per id (a Series indexed by id)
df["col"].unique(); df["col"].value_counts()
```

1.11 Series indexing quirks (from the Pandas notebook)

```
s = pd.Series([1, 3, 5, 7, 9])
s[0]; s[1:3]; s[::-1]; s[[1, 3]]; s[[True, False, True, True, False]]
# s[-1]  -> ERROR (no label -1).  Use s.iloc[-1]
s = pd.Series([1, 3, 5, 7, 9], [1, 2, 3, 4, 5])   # integer labels that are not 0..4
s[3]        # -> 5   (LABEL 3)
s[1:3]      # -> 3, 5 (POSITIONS 1, 2) -- confusing! so always use loc/iloc
s.loc[1:3]  # -> 1, 3, 5 (labels 1..3 inclusive)
s.iloc[1:3] # -> 3, 5
```

1.12 statsmodels regression (HW4)

```
import statsmodels.formula.api as smf
model1 = smf.ols("lcrime ~ lpolice + lemp + lun + lymale + lwhite", data=fulldata).fit()
model2 = smf.ols("dlcrime ~ dlpolice + dlemp + C(week)", data=diffdata).fit()  # C() = dummies (fixed effects)
model1.params; model1.bse; model1.rsquared; model1.summary(); model1.predict()
```

Why the log-log regression: coefficients are elasticities, and an extra police hour matters less when there are already many. Why difference year 2 minus year 1: it removes borough effects that don't change over time. Week fixed effects pick up shocks common to all boroughs in a given week. The cross-sectional coefficient on police is biased upward by reverse causation (high-crime boroughs get more police).

2. Scope and mutability: the trace method (two 8-point questions)

The five facts that answer every trace question

1. **Assignment inside a function (`x = ...`) creates a LOCAL variable.** The global with the same name is untouched.
2. **Parameters are local names bound to the objects passed in.** `f(L, x)` binds the 1st parameter to `L`'s object and the 2nd to `x`'s object, *whatever the parameters are called.*
3. **Mutating a passed-in mutable object (`L[0]=8`, `L.append(...)`) changes the caller's object.** Re-assigning the name (`L = 3`) does not.
4. **Slices and arithmetic create NEW objects**: `x[::2]`, `x[1:]`, `x + [5]`, `x.copy()`. `y = x` does not copy.
5. **Code after `return` never runs** (it's dead code). Exams hide lines such as `x=x-5` or `x[1]=8` after the `return`, inside the function.

Method. Keep two boxes on scratch paper, GLOBAL and LOCAL. When the function is called, write each parameter in LOCAL with an arrow to the object it refers to. Run the body line by line, applying the facts above. When the function returns, erase LOCAL. Ints, floats, strings, and tuples are immutable, so "changing" one always makes a new object.

Trap: using a global you also assign to

```
z = 3
def g(x):
    z = z + 1      # UnboundLocalError: z is local (assigned in g) but used before assignment
    return x * z
```

Reading a global without assigning to it works: `def h(x): return x*y` uses global `y`.

Copy vs. deepcopy. `c = l.copy()` is a new outer list, but any inner lists are *shared*: `c[2].append(5)` also changes `l`. `copy.deepcopy(l)` copies everything. `a is b` asks "same object?"; `a == b` asks "same value?".

Default arguments: `def myvar(values, sample=True)`. The default is used if no argument is passed, and it overrides a global with the same name. Named arguments can come in any order: `myvar(sample=False, values=a)`.

3. Core Python you must not fumble

```
7 // 2 -> 3      7 % 2 -> 1      2 ** 3 -> 8      7 / 2 -> 3.5      -7 // 2 -> -4
float("inf"), -float("inf")      # start values for min / max searches
66 // 12 -> 5 (feet)     66 % 12 -> 6 (inches)      n % 3 == 0  -> divisible by 3
```

Iterables. `str` and `tuple` are immutable; `list` is mutable; `range(start, stop, step)` excludes `stop`.

```
s = "Go Mustangs!"
s[3:5] -> "Mu"   s[-2] -> "s"   s[::-1] -> reversed   s[3:1:-1] -> "M "   s[1:4:2] -> "o"+"M"
list(range(10, 0, -2)) -> [10, 8, 6, 4, 2]          t = (3,)   # one-element tuple needs the comma
l.append(v)  l.insert(i, v)  l.remove(v)  # removes first matching VALUE
del l[i]     # removes by INDEX
[3, 4] + [1] -> [3, 4, 1]        "ell" in "hello" -> True        len("") -> 0
```

4. Loop templates (copy these patterns)

```
# factorial of x (x >= 1), no built-ins
y = 1
for i in range(1, x + 1):
    y = y * i
# product (p) / sum of list x
p = 1
for v in x:
    p = p * v
# min with index
cur = float("inf")
for i in range(len(l)):
    if l[i] < cur:
        cur = l[i]; idx = i
# second highest (one pass)
def f(x):
    m = -float("inf"); m2 = -float("inf")
    for v in x:
        if v > m:
            m2 = m; m = v        # old max becomes second
        elif v > m2:
            m2 = v
    return m2
# filter + transform into a NEW list (input not modified)
def f(x):
    r = []
    for v in x:
        if v >= 0:
            r.append(v ** 2)
    return r
# list comprehension version
r = [v ** 2 for v in x if v >= 0]
# cumulative sum (new list)
r = [l[0]]
for i in range(1, len(l)):
```

```
    r.append(r[i - 1] + l[i])
# move first element to end (new list)
def f(x):
    r = x[1:]          # slice = new list, so x is safe
    r.append(x[0])
    return r
# dict return
def inches_to_feet(n):
    return {"feet": n // 12, "inches": n % 12}
# list of dicts
s = x[0]["c"] + x[len(x) - 1]["j"]      # or x[-1]["j"]
```

while vs for: use `for` when you know how many iterations there are, and `while` until a condition holds (make sure something in the body changes the condition). `break` exits the loop; `continue` skips to the next iteration.

5. Dictionaries (hash tables)

```
d = {}; d["x"] = 3; d["y"] = 6        # add / overwrite
d["y"]          # 6        d["z"] -> KeyError
"x" in d        # True   (checks KEYS)       6 in d.values()  # True
d.keys(); d.values(); len(d)
d2 = {"a": 4, "b": "hello", "c": [1, 2, 6], 4: 3.14}
for k in d:                        # loops over keys
    print(k, d[k])
counts = {}                        # counting pattern
for v in data:
    if v in counts: counts[v] = counts[v] + 1
    else:           counts[v] = 1
```

Keys must be immutable (int, float, str, bool, tuple). Lookup is $O(1)$ on average; searching a list is $O(n)$.

6. NumPy

```
np.array([1, 3, 5]); np.zeros(20); np.zeros(20, dtype=int); np.ones(10); np.full(10, 4)
np.arange(0, 10, 1)      # 0..9 (stop excluded)
np.linspace(0, 1, 101)   # 101 points, 0 and 1 INCLUDED  (grid search!)
np.random.seed(100); np.random.random(10); np.random.normal(0, 1, 10); np.random.randint(0, 100, 10)
np.random.choice(np.arange(30), 10, replace=False)
a = np.arange(1, 101)    # HW3: no loops
b = a ** 2;  c = a * (a + 1) / 2;  d = (c % 3 == 0)
e = np.nonzero(d)[0]     # INDICES where True (nonzero returns a tuple -> take [0])
f = c[d]                 # VALUES where True (boolean mask)
g = np.cumsum(b)
np.sum, np.prod, np.min, np.max, np.mean, np.std, np.var, np.median, np.sort, np.argmin, np.argmax
np.any(x > 3); np.all(x > 3); np.sum(x > 3)          # count of True
x[(x >= 3) & (x <= 5)]; x[~(x > 3)]; x[np.array([4, 1, 3])]   # fancy indexing
np.concatenate([x, y])
m = np.arange(6).reshape(2, 3); m.shape -> (2, 3); m[1, 2]; m[:, 1]; m[0]
```

Vectorized operations (`x + y`, `2*x`, `np.log(x)`) are much faster than Python loops. Arrays hold a single type and are mutable.

7. Functions and classes

```
def f(a, b=2):          # body runs only when f is CALLED
    return a + b, a * b # returns a tuple; unpack with s, p = f(3)
class Point:
    def __init__(self, x=0, y=0):     # constructor; self = the instance
        self.x = x; self.y = y
    def dist(self):                   # method: first parameter is self
        return (self.x ** 2 + self.y ** 2) ** 0.5
    def __str__(self):                # used by print(p)
        return "(" + str(self.x) + "," + str(self.y) + ")"
```

```
    def __add__(self, other):         # overloads p + q
        return Point(self.x + other.x, self.y + other.y)
p = Point(3, 4); p.dist() -> 5.0
```

Objects are mutable: a function that does `p.x = p.x + 1` changes the caller's object. A **method** is called on an object (`df.drop(...)`, `l.append(...)`); a **function** is called on its own (`pd.concat(...)`, `len(l)`).

Graph (HW6). `Graph.nodes` is a dict mapping name to `Node`. Each `Node` has a name and its edges, and each edge has a weight and two nodes. For an undirected graph, `add_edge(n1, n2, w)` adds the edge to *both* nodes, overwrites an existing weight, and does nothing if a name is invalid. `get_node(n)` returns -1 if `n` is missing.

8. Big-O, Ω, Θ

Definitions and rules

O = worst case (upper bound); Ω = best case (lower bound); Θ = both, when best and worst cases are the same order. Drop constants and lower-order terms: $2n^2 + 2n + 10 \to O(n^2)$. Use the right n: the input size that can grow.

Code pattern	Cost
Fixed number of operations, or `return` on the first iteration	$O(1)$
`while n>1: n = n//2` (halve each time), binary search, fast power	$O(\log n)$
One loop over n items, `np.mean`, `sum`, `in` on a list	$O(n)$
Nested loops over n and n (e.g. an $n \times n$ matrix), insertion sort worst case	$O(n^2)$
Loop calling an $O(n)$ function each time (`np.mean` inside a loop)	$O(n^2)$ (move it out of the loop!)
Grid search with n decimal places in 1-D / 2-D	$O(10^n)$ / $O(100^n)$, worse than any polynomial

Ordering: $1 < \log n < n < n \log n < n^2 < n^{10} < 2^n < 10^n$. (So $O(10^n)$ is *much worse* than $O(n^{10})$.)

Search with early return: $\Omega(1)$ (found at the first position) and $O(n)$ (not there), or $O(n^2)$ for a matrix. **Binary search:** $\Omega(1)$ (middle element on the first check) and $O(\log n)$.

9. Algorithms you might have to code from pseudocode

```
def inssort(x):                       # insertion sort (practice Q5)
    r = x.copy()                      # don't modify input
    for i in range(1, len(r)):        # i = 1 .. n-1
        j = i
        while j >= 1 and r[j] < r[j - 1]:
            temp = r[j]; r[j] = r[j - 1]; r[j - 1] = temp    # swap
            j = j - 1
    return r
# best case (already sorted): while never runs -> Omega(n)
# worst case (reverse sorted): while runs i times -> 1+2+...+(n-1) = n(n-1)/2 -> O(n^2)

x = np.linspace(0, 1, 101); y = np.linspace(0, 1, 101)    # 2-D grid search for a MAX
max_value = -float("inf")
for i in x:
    for j in y:
        fv = f(i, j)
        if fv > max_value:
            max_value = fv; x_max = i; y_max = j

ans = n / 2; delta = ans / 2                    # bisection: sqrt(n)
while abs(ans ** 2 - n) > e:
    if ans ** 2 > n: ans = ans - delta
    else:            ans = ans + delta
    delta = delta / 2

x = 0                                           # Newton-Raphson minimization
while abs(f1(x)) > eps:                         # f1 = f', f2 = f''
    x = x - f1(x) / f2(x)
```

```
def power2(x, n):                              # fast power, O(log n)
    r = 1
    while n > 0:
        if n % 2 > 0: r = r * x
        n = n // 2; x = x * x
    return r
```

Translation tips: "from $i = 1$ to $n - 1$ inclusive" is `range(1, n)`. "Repeat as long as A AND B" is `while A and B:`. Put the index check *first* (`j >= 1 and r[j] < r[j-1]`) so `r[-1]` is never evaluated. "Return a new list" means work on `x.copy()`.

10. Practice midterm answers, worked (all verified by running them)

Fall 2025 Q6. `x=[0,1]; n=7`; `f(n)` makes a LOCAL `x=[0,1]`. The loop runs with $n = 7$ (append 15, $n \to 3$), then $n = 3$ (append 7, $n \to 1$), then stops. (a) `y = [0,1,15,7]`. (b) Global `x = [0,1]` (the function only changed its local `x`). (c) `n = 7` (the function changed only its local `n`). (d) n halves each time, so $O(\log n)$.

Fall 2025 Q7. `L=[4,3,2,1]; x=5`. Calling `f(L, L[1])` binds local `x` to the global `L` list and local `y` to 3. Then `y=6`; local `L = x[::2] = [4,2]` (a new list); `L[0]=6` gives local `[6,2]`; `x.append(6)` mutates the *global* list L to `[4,3,2,1,6]`; return local `L[1] = 2`. The line `x[1]=8` is after the `return`, so it is dead code. (a) 2. (b) `y = 2*5 = 10`. (c) `x = 5`. (d) `L = [4,3,2,1,6]`.

Fall 2025 Q8. `search_matrix`: $O(n^2)$, $\Omega(1)$. `search_sorted_array`: $O(\log n)$, $\Omega(1)$. **Q5:** insertion sort is $\Omega(n)$ and $O(n^2)$ (code in §9). **Q1–Q3:** templates in §4. **Q4:** §1.9.

Fall 2023 Q5. The function's `r` and `x` are local: (a) `y = 9/3 = 3.0`, (b) `r = 10`, (c) `x = 2`, (d) $O(n)$.

Fall 2023 Q6. `x=[1,2,3,4]; L=4; y=1`. Calling `f(L, x)` binds parameter `y` to 4 and parameter `L` to the global *list* `x`. Then `L[0]=8` mutates the global list to `[8,2,3,4]`; `y=3`, `x=2`, and `L=x` are all local; return 3. `x=x-5` comes after the `return`, so it is dead code. (a) 3. (b) `y = 1 + 3*1 = 4`. (c) `x = [8,2,3,4]`. (d) `L = 4`.

Fall 2023 Q5 (grid search). Code in §9. (b) With n decimal places there are $10^n + 1$ points per axis, so $(10^n)^2$ evaluations: $O(100^n)$, which the key writes as $O(10^n)$ (exponential). (c) Much worse than $O(n^{10})$.

Practice quiz trace. `x=[0,1]; n=7`: $n = 7$: append 15, $7\%3 \neq 0$, $n \to 3$. $n = 3$: append 7, $3\%3 = 0$ so `x[0]=3`, $n \to 1$. Result: `n = 1`, `x = [3,1,15,7]`.

11. Last-five-minutes checklist

1. "Don't modify the input": start with `r = x.copy()` or build `r = []`. Never write `r = x`.
2. "No built-ins": no `sum`/`max`/`min`/`sorted`/`abs`. Use loops with `float("inf")`.
3. DataFrames: use `.iloc[i]` for positions, `&`/`|` with parentheses, reassign after `drop`/`concat`/`sort_values`, and `reset_index(drop=Tru` before subtracting two filtered halves.
4. Trace questions: find dead code after `return`; separate mutation (`L[0]=`, `.append`) from re-binding (`L=`); match arguments to parameters by *position*.
5. Big-O: halving gives $\log n$, nested loops give n^2, an early `return` gives $\Omega(1)$, and decimal precision gives 10^n.
6. Indent carefully on paper. The `return` goes *after* the loop, not inside it.